Loopless Generation of Multi-ary Trees*

Ro-Yu Wu[1] Jou–Ming Chang[2] Yue-Li Wang[3,†]

[1] Department of Industrial Engineering Management, Lunghwa University of Science and Technology, Taoyuan, Taiwan, ROC (eric@mail.lhu.edu.tw)

[2] Department of Information Management, National Taipei College of Business, Taipei, Taiwan, ROC (spade@mail.ntcb.edu.tw)

[3] Department of Computer Science and Information Engineering, National Chi Nan University, Nantou, Taiwan, ROC (yuelwang@ncnu.edu.tw)

Abstract

In this paper, we use a concise representation, called right-distance sequences (or RD-sequences for short), to describe all multi-ary trees with n internal nodes. An ordered tree is called a multi-ary tree if every node has it own maximum number of sons. Using a coding tree, a systematic way can help us to investigate the structural representation of multi-ary trees. Consequently, we present a loopless algorithm to generate Gray-codes of multi-ary trees using RD-sequences.

Keywords: Multi-ary trees; Coding trees; Gray-code generation; Loopless algorithm;

1. Introduction

A tree is said to be *ordered* if the sons of each internal node can be distinguished as the first son, the second son and so on. An ordered tree of n internal nodes is called a *multi-ary tree* if every node has it own maximum number of sons. Multi-ary trees were mentioned in [8] and the author used the name *non-regular trees*. For the case where every node has the same maximum number of sons, a multi-ary tree is called a *t-ary tree*. A t-ary tree with $t = 2$ is the so-called *binary tree*.

Generating all n nodes t-ary trees has been extensively studied [1–6, 9–21, 23–32, 34–37, 39–47]. An important issue on enumerating various classes of objects is that assuring each object can be generated in a constant time. Therefore, a number of algorithms have been proposed to generate t-ary trees with constant amortized time (i.e., constant average time per tree) [2, 9, 15, 27, 31, 35, 41, 42, 45, 46]. To emphasize the constant time generation, in [7], Ehrlich defined that a *loopless generation algorithm* is an algorithm where the amount of computations needed to transform one object into the next takes only $O(1)$ time. Thus, a loopless algorithm can be implemented by using, after the initialization, only assignment statements and the control statement "if-then-else" and no recursion allowed.

For a loopless algorithm, the number of changes between two successive tree sequences must be bounded by a constant. In [28], the authors gave a loopless algorithm for generating binary tree sequences in which the num-

*This research was partially supported by National Science Council under the Grants NSC96-2221-E-260-018.

†Corresponding author.

ber of changes between two successive tree sequences is 2. In [11], the difference between two successive t-ary tree sequences is also bounded by 2 positions. When the difference between two successive t-ary tree sequences can only occur at one position, it is well-known that the sequences are changed in Gray-code order. An excellent survey for listing combinatorial objects in Gray-code order can refer to Savage [33]. Loopless algorithms for generating binary trees or t-ary trees take a constant amount of time in the worst case and have attracted a great deal of attention in recent literature [13–15, 18, 29, 36, 37, 41–43].

The first loopless Gray-code generation algorithm was given by Williamson [38]. Applying Williamson's algorithm, Vajnovszki [36] presented a loopless algorithm to generate Gray-codes of binary trees using the so-called W-sequences (i.e., the left weight sequences) introduced by Pallo [22]. Inspired by the idea from [36], Korsh and LaFollette [13] generalized the W-sequences and proposed a loopless Gray-code generation for t-ary trees, where the size of space (i.e., the number of arrays of size n) required by the algorithm depends on t. Using a shift operation between two t-ary trees, Korsh and Lipschutz [14] developed another loopless algorithm to list t-ary trees. Takaoka [34] proposed a loopless algorithm to generate Gray-codes of binary trees using the T-sequences (i.e., the characteristic sequences). Vajnovszki [37] has presented a loopless algorithm to generate Gray-codes of binary trees using the P-sequences introduced by Pallo [24]. Later on, using the well-formed integer sequences suggested by Zaks [45], i.e., Z-sequences, an implementation that uses a fixed number of arrays to generate Gray-codes of t-ary trees was developed by Roelants van Baronaigien [29]. Meanwhile, Xiang et al. [41, 42] provided another loopless algorithm with a slight improvement under the same representation. Compared with the loopless algorithms to generate Gray codes for binary trees in [36] and for t-ary trees in [13], their loopless algorithm is more efficient and also conceptually simpler than those of the predecessors. The above researches focus on t-ary trees ($t \geqslant 2$), i.e., each node has t children in regular trees. In this paper, we propose a loopless algorithm which generates Gray-codes of multi-ary trees. Thus, the enumeration of Gray-codes of binary trees and t-ary trees can also be achieved by our algorithm. Table 1 lists the aforementioned papers.

The remaining part of this paper is organized as follows. In Section 2, we introduce a structural representation of multi-ary trees. In Section 3, we propose a loopless generation algorithm for generating all multi-ary tree sequences in Gray-code order. Finally, our conclusion is contained in Section 4.

Table 1: Loopless tree generation algorithms

Papers	Tree Types	Used Codes	Successive Changes
Baronaigien [28]	binary	rotation codewords	2
Korsh [11]	t-ary	rotation codewords	2
Vajnovszki [36]	binary	W-sequences	1
Korsh and LaFollette [13]	t-ary	W-sequences	1
Korsh and S. Lipschutz [14]	t-ary	W-sequences	1
Takaoka [34]	binary	T-sequences	1
Vajnovszki [37]	binary	P-sequences	1
Roelants van Baronaigien [29]	t-ary	Z-sequences	1
Xiang et. al. [41, 42]	t-ary	Z-sequences	1
This paper	Multi-ary	RD-sequences	1

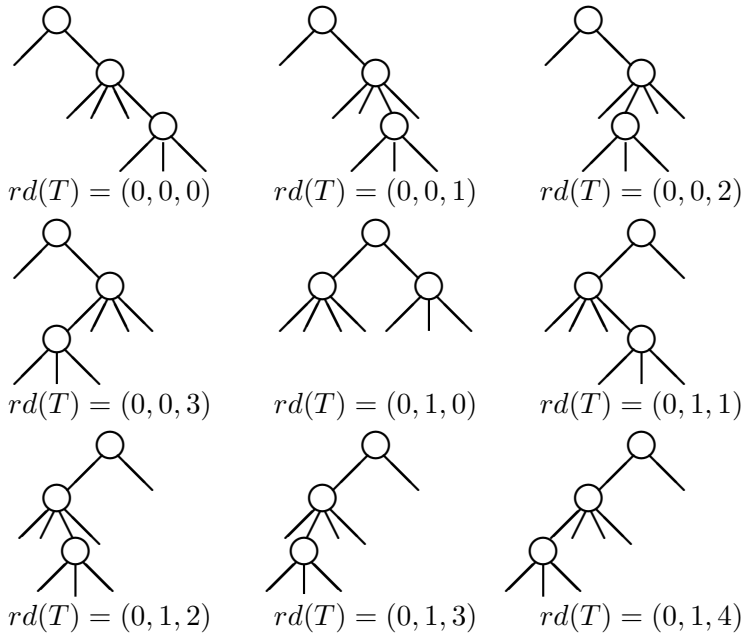

Figure 1: All multi-ary trees with three internal nodes and $S = (2, 4, 3)$.

2. A Structural Representation of Multi-ary Trees

In this section, we begin with a structure representation for Multi-ary trees. Let T be a multi-ary tree with n internal nodes numbered from 1 to n by the preorder traversal order (i.e. visit recursively the root and then the subtrees of T from left to right). Let $s_i \geqslant 2$ denote the maximum number of sons of node i. The sequence $S = (s_1, s_2, \ldots, s_n)$ is called the *branching sequence* of T. For each internal node $i \in T$, we define *right distance* d_i as follows. If $i = 1$, i.e., the root, then $d_i = 0$. For every nonroot node i, $d_i = d_{p(i)} + s_{p(i)} - s$, where $p(i)$ is the parent of node i and node i is the s-th son (from left to right) of node $p(i)$. Thus, $d_i = d_{p(i)}$ when node i is the rightmost child of its parent. A multi-ary tree T can be represented by the sequence $rd(T) = (d_1, d_2, \ldots, d_n)$ and we call $rd(T)$ the *right distance sequence* (abbreviated as RD-sequence) of T. For example, Figure 1 shows RD-sequences of all the possible multi-ary trees with three internal nodes and $S = (2, 4, 3)$.

Lemma 1 *Let $S = (s_1, s_2, \ldots, s_n)$ be the branching sequence of multi-ary tree T and $rd(T) = (d_1, d_2, \ldots, d_n)$. Then, $0 \leqslant d_i \leqslant d_{i-1} + (s_{i-1} - 1)$ for $i = 2, 3, \cdots, n$.*

Proof. If node $i - 1$ is the parent of node i, then, by definition, we can obtain the following derivation.

$$\begin{aligned} d_i &= d_{p(i)} + s_{p(i)} - s \\ &= d_{i-1} + s_{i-1} - s \\ &\leqslant d_{i-1} + s_{i-1} - 1, \end{aligned}$$

where $p(i)$ is the parent of node i and node i is the s-th son of node $p(i)$.

For the case where node $i - 1$ is not the parent of node i, there exists an ancestor, say node x, of node $i - 1$ which is also a sibling of node i. By definition again, we can derive that $d_i < d_x \leqslant d_{i-1} < d_{i-1} + s_{i-1} - 1$. □

A synonym of RD-sequence is called the *inversion table* in [21]. In this paper, we rename it as "RD-sequence" to indicate the

right distances of nodes in a multi-ary tree. Let $\mathscr{T}_{n,S}$ denote the set of all multi-ary trees with n internal nodes and branching sequence $S = (s_1, s_2, \ldots, s_n)$. For easily understanding our loopless algorithm, we use a systematic way to describe all multi-ary trees in $\mathscr{T}_{n,S}$. A *multi-ary coding tree* with n levels is a rooted tree $\mathbb{T}$ constructed by the following rules:

(1) Initially, a root with label 0 is created for $\mathbb{T}$ in level 1;

(2) For every nonleaf node (i.e., a node not in the n-th level) with label l in the newly created level i, create s_i+l sons for it and label these sons by $0, 1, \ldots, s_i+l-1$ from left to right in the next level of $\mathbb{T}$.

Consequently, the full labels along the path from the root to a leaf in $\mathbb{T}$ represent the RD-sequence of a multi-ary tree with n internal nodes. Figure 2 demonstrates the multi-ary coding tree with three levels and shows all RD-sequences for multi-ary trees with three nodes and $S = (2, 4, 3)$.

By observing the construction of $\mathscr{T}_{n,S}$, we use a formula to describe the number of all possible multi-ary trees of a given branching sequence S.

Theorem 2 *Let $\mathscr{T}_{n,S}$ denote the set of all multi-ary trees with n internal nodes and branching sequence $S = (s_1, s_2, \ldots, s_n)$. The cardinality of $\mathscr{T}_{n,S}$ is as follows:*

$$|\mathscr{T}_{n,S}| = \sum_{k_1=0}^{s_1-1} \sum_{k_2=0}^{k_1+s_2-1} \cdots \sum_{k_{n-2}=0}^{k_{n-3}+s_{n-2}-1} (k_{n-2}+s_{n-1}).$$

Similar concepts of multi-ary coding trees were independently introduced under various names including the backtracking tree [47], the recursion tree [18], and the coding tree [40]. Backtracking trees and recursion trees are used to generate binary tree sequences while coding trees are constructed for the regular t-ary trees. However, all homologues can be obtained from the multi-ary coding tree constructed above by setting all $s_i \in S$ to 2 for binary trees or t for t-ary trees. By using a backtracking technique on the multi-ary coding tree of $\mathscr{T}_{n,S}$, we can easily generate all RD-sequences in $\mathscr{T}_{n,S}$. However, it can be found that the generated sequences are not in Gray-code order. In the next section, we shall propose a way to rearrange the multi-ary coding tree so that a Gray-code order enumeration can be obtained.

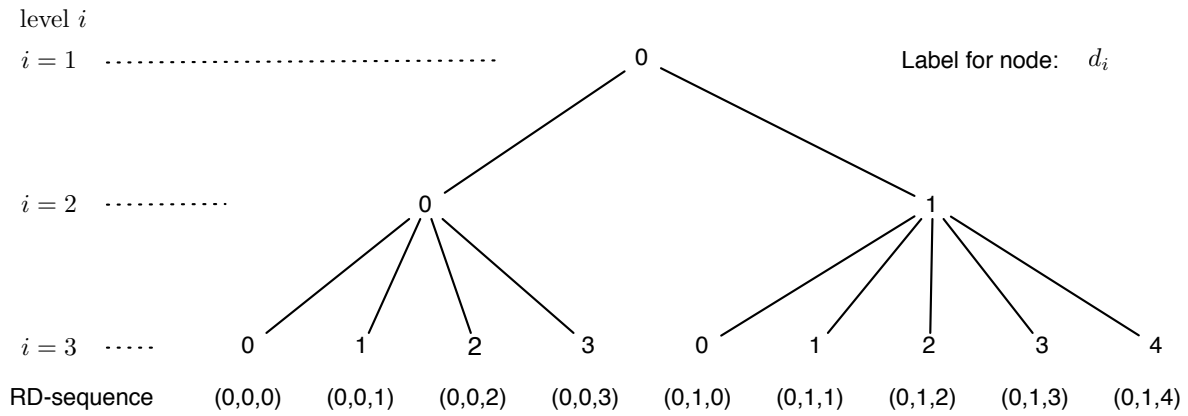

Figure 2: The multi-ary coding tree and RD-sequences of 3-node trees with $S = (2, 4, 3)$.

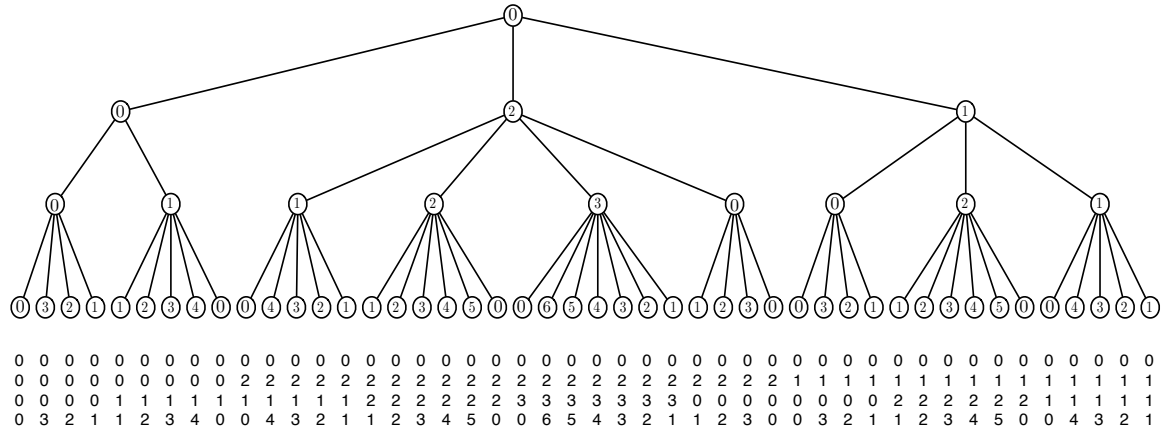

Figure 3: A flip-flap tree and the enumeration of RD-sequences for 4-node multi-ary trees with $S = (3, 2, 4, 3)$ in Gray-code order.

3. Loopless Gray-code Generation for Multi-ary Trees

In this section, using RD-sequence representation, we propose a loopless Gray-code generation algorithm for multi-ary trees.

Our basic idea is that, for every nonleaf node x in level i of $\mathbb{T}$, all the labels of its $l+s_i$ sons are arranged in either $1, 2, \ldots, l+s_i-1, 0$ or $0, l+s_i-1, \ldots, 2, 1$, where l is the label of node x. Such an arrangement is called an *up fragment* for the former case and a *down fragment* for the latter case. If the sons of node i are arranged in up fragment, then the sons of its adjacency siblings, i.e., node $i-1$ and node $i+1$, if exist, are arranged in down fragment and vice versa. We call such an arrangement a *flip-flap* arrangement and the resulting multi-ary coding tree a *flip-flap multi-ary coding tree* (or a *flip-flap tree* for short). In a flip-flap tree, the role of RD-sequences is born with a compact spectrum. In view from a flip-flap tree $\mathbb{T}$, if the nodes are labeled by RD-sequences, then for any fragment of $\mathbb{T}$, the two particular labels 0 and 1 always appear in its spectrum, and thus they can be used as the boundaries in this fragment. Then, the two types of fragments appear alternately in each level of $\mathbb{T}$. By this way, if the full labels along the path in the left-arm of $\mathbb{T}$ are determined, the framework of $\mathbb{T}$ is also determined instantly. Since there are 2^{n-1} choices to label the nodes in the left-arm of $\mathbb{T}$ (recall that $d_1 = 0$), it ensures that the number of different ways of Gray-code generations for multi-ary trees with n internal nodes in RD-sequences is at least 2^{n-1}. For instance, Figure 3 shows a Gray-code enumeration for $n = 4$, where RD-sequences begin with $(0, 0, 0, 0)$ and thus the initial fragment in each level of $\mathbb{T}$ is given by a down fragment.

Collecting all labels from the root to a leaf in a flip-flap tree, we can obtain an RD-sequence. Thus, a leaf of a flip-flap tree can represent a unique RD-sequence. The following lemma provides the base for our loopless algorithm.

Lemma 3 *There is only one position having different digits for the RD-sequences obtained from two adjacency leaves in a flip-flap tree* $\mathbb{T}$.

Proof. Let $rd(T_1)$ and $rd(T_2)$ be RD-sequences of any two adjacency leaves x and y, respectively, in flip-flap tree $\mathbb{T}$. Let node

z be the lowest common ancestor of leaves x and y in the flip-flap tree $\mathbb{T}$. It is clear that labels collected from the root to node z are the same for leaves x and y and different for z's sons. By the flip-flap arrangement property, the remaining labels of leaves x and y must also be the same. Thus, there is only one position having different digits for $rd(T_1)$ and $rd(T_2)$. □

Since a flip-flap tree only exists conceptually, a backtracking technique cannot be applied directly on flip-flap tree $\mathbb{T}$. Now, we give the implementation in detail. A procedure called `NextRD()` is developed for generating the next RD-sequence. Let array $s[1..n]$ be the branching sequence S and array $d[1..n]$ be used for storing the current RD-sequence in Gray-code order. A global variable i indicates the next processing position of array d. Two arrays $c[1..n]$ and $F[1..n]$ are employed to record auxiliary information. Each $c[i]$ indicates the type of current fragment at level i of $\mathbb{T}$, where $c[i]$ is set to 1 for up fragment and -1 for down fragment. Array F is used for keeping track of positions to be processed. In the beginning, $d[j]$, $c[j]$, and $F(j)$ for $1 \leqslant j \leqslant n$ are initialized to 0, -1 (down fragment), and j, respectively. Also, set $i = n$ initially. Now, procedure `NextRD()` can be described as follows.

1. When `NextRD()` is invoked, according to the type of current fragment, it first updates $d[i]$ by the setting $d[i] = (d[i] + c[i])\%(d[i-1] + s[i-1])$, where % stands for the modulo operation.

2. If the fragment is not on a boundary, the next change will occur in position n (i.e., $i = n$).

3. If the fragment is in face of a boundary, then it performs the following:

   - Change the type of current fragment (i.e., $c[i] = -c[i]$);
   - Propagate the information of F from an upper level to a lower level in $\mathbb{T}$ (i.e., $F[i] = F[i-1]$);
   - Recover the information of F for the upper level of $\mathbb{T}$ (i.e., $F[i-1] = i - 1$);
   - Let $F[n]$ be the position to be processed (i.e., $i = F[n]$);
   - Retrieve the initial value of $F[n]$ (i.e., $F[n] = n$);

4. When $i = 1$, all RD-sequences have been obtained.

Note that a modulo operation requires more clocks (execution time) than a comparison operation and/or an assignment operation in almost modern computers. To make a fair competition with [29] and [41], we do not use a modulo operation in the setting of $d[i]$. It is easy to see that the use of two comparisons (one is for determining the type of current fragment and the other is for testing the extreme value of $d[i]$) and one assignment (for updating $d[i]$) suffice to stand for the modulo operation. Since two extreme values of $d[i]$ are 0 and $d[i-1] + s[i-1] - 1$, the change can be done by setting $d[i]$ to an extreme value (by adding $c[i]$) if it encounters with another extreme value. On the other hand, the value of $d[i]$ is normally increased or decreased by one according to the type of $c[i]$. Furthermore, to determine whether the fragment is faced with a boundary, this can be done by checking $d[i] \leqslant 1$. However, we can omit the checking for an up fragment since the previous setting $d[i] = 0$ means that the fragment has arrived at a boundary. As the loopless algorithms have presented in [13, 29, 36, 41], procedure `NextRD()` is again inspired from Williamson's algorithm [38], and thus its correctness directly follows. For accuracy, we give the flowchart of `NextRD()` in Figure 4. Obviously, our algorithm needs $4n+O(1)$ space. Also, it is easy to verify that 2/3 comparisons and 2/6 assignments are executed in the best/worst cases between two successive sequences.

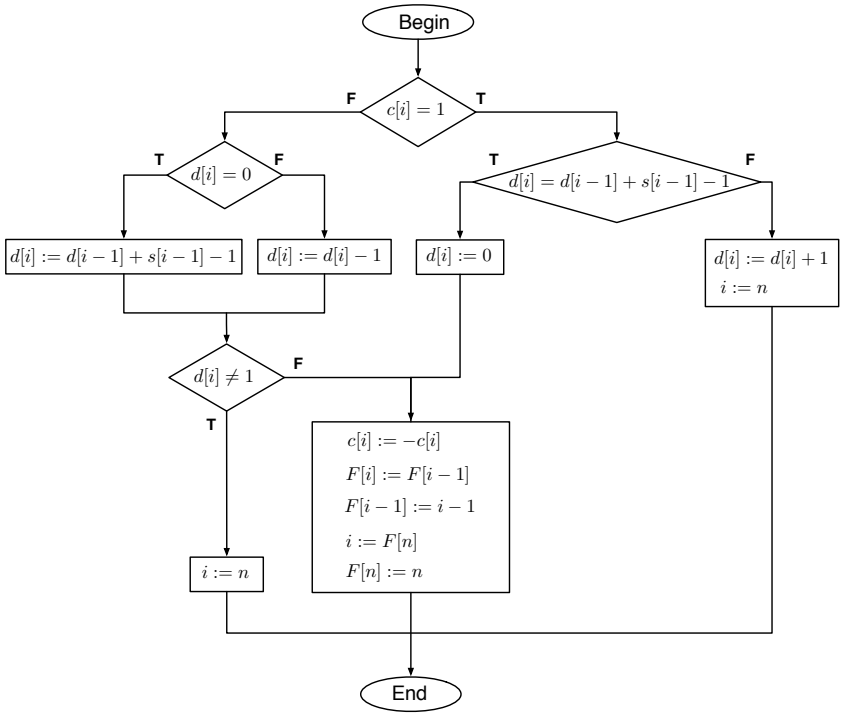

Figure 4: The flowchart of procedure `NextRD()`.

A complete implementation of `NextRD()` written by C++ is as follows.

```
void NextRD() {
    if (c[i] == 1)
        if (d[i] == d[i-1]+s[i-1]-1)
            d[i] = 0;
        else
        {
            d[i] = d[i]+1;   i = n;
            return;
        }
    else
    {
        if (d[i] == 0)
            d[i] = d[i-1]+s[i-1]-1;
        else
            d[i] = d[i]-1;
        if (d[i] != 1)
        {
            i = n; return;
        }
    }
    c[i] = -c[i];  F[i] = F[i-1];
    F[i-1] = i-1;  i = F[n];
    F[n] = n;
}
```

We summarize our result as the following theorem.

Theorem 4 *All multi-ary trees with branching sequence $S = (s_1, s_2, \ldots, s_n)$ and n internal nodes can be generated in Gray-code order in $O(|\mathscr{T}_{n,S}|)$ time and generating each RD-sequence takes only constant time.*

4. Conclusion

In this paper, we use RD-sequences to represent multi-ary trees. This representation is not only conceptually simple, but it also has a compact spectrum. Using RD-sequences to label the nodes of a coding tree, we can describe all multi-ary trees in a systematic way. For the Gray-code generation of multi-ary trees, we design a loopless algorithm that uses $4n$+O(1) space. To generate Gray-codes of binary trees and t-ary trees can also be achieved by our algorithm.

References

[1] H. Ahrabian and A. Nowzari-Dalini, On the generation of binary trees in A-order, *International Journal of Computer Mathematics* **71** (1999), 1–7.

[2] H. Ahrabian and A. Nowzari-Dalini, Generation of t-ary trees with Ballot-sequences, *International Journal of Computer Mathematics* **80** (2003), 1243–1249.

[3] H. Ahrabian and A. Nowzari-Dalini, On the generation of P-sequences, *Advanced Modeling Optimization*, **5** (2003), 27–38.

[4] H. Ahrabian and A. Nowzari-Dalini, Gray code algorithm for listing k-ary trees, *Parallel Computing*, **31** (2005), 948–955.

[5] H. Ahrabian and A. Nowzari-Dalini, Parallel generation of binary trees in A-order, *Studies in Informatics and Control*, **13** (2004), 243–251.

[6] S. G. Akl and I. Stojmenović, Generating t-ary trees in parallel, *Nordic Journal of Computing* **3** (1996), 63–71.

[7] G. Ehrlich, Loopless algorithms for generating permutations, combination, and other combinatorial configurations, *Journal of the ACM* **20** (1973), 500–513.

[8] M. C. Er, A simple algorithm for generating non-regular trees in lexicographic order, *The Computer Journal* **31** (1988), 61–64.

[9] M. C. Er, Efficient generation of k-ary trees in natural order, *The Computer Journal* **35** (1992), 306–308.

[10] G. Knott, A numbering system for binary trees, *Communication of the ACM*, **20** (1977), 113–115.

[11] J. F. Korsh, Loopless generation of k-ary tree sequences, *Information Processing Letters* **52** (1994), 243–247.

[12] J. F. Korsh, A-order generation of k-ary trees with $4k-4$ letter alphabet, *Journal of Information and Optimization Science* **16** (1995), 557–567.

[13] J. F. Korsh and P. LaFollette, Loopless generation of Gray code for k-ary trees, *Information Processing Letters* **70** (1999), 7–11.

[14] J. F. Korsh and S. Lipschutz, Shifts and loopless generation of k-ary trees, *Information Processing Letters* **65** (1998), 235–240.

[15] J. F. Korsh, Generating t-ary trees in linked representation, *The Computer Journal* **48** (2005), 488–497.

[16] C. C. Lee, D. T. Lee and C. K. Wong, Generating binary trees of bounded heighted, *Acta Informatica*, **23** (1986), 529-544.

[17] J. M. Lucas, The rotation graph of binary trees is Hamiltonian, *Journal of Algorithms* **8** (1987), 503–535.

[18] J. M. Lucas, D. Roelants van Baronaigien, and F. Ruskey, On rotations and the generation of binary trees, *Journal of Algorithms* **15** (1993), 343–366.

[19] E. Mäkinen, Left distance binary tree representations, *BIT* **27** (1987), 163–169.

[20] E. Mäkinen, Generating random binary trees – A survey, *Information Sciences* **115** (1999), 123–136.

[21] H. W. Martin and B. J. Orr, A random binary tree generator, in: *Proc. of the 17th conference on ACM Annual Computer Science Conference*, Louisville, Kentucky, February, 1989, 33–38.

[22] J. Pallo, Enumerating, ranking and unranking binary trees, *The Computer Journal* **29** (1986), 171–175.

[23] J. Pallo, Generating trees with n nodes and m leaves, *International Journal of Computer Mathematics* **21** (1987), 133–144.

[24] J. Pallo and R. Racca, A note on generating binary trees in A-order and B-

order, *International Journal of Computer Mathematics* **18** (1985), 27–39.

[25] A. Proskurowski and F. Ruskey, Binary tree Gray codes, *Journal of Algorithms* **6** (1985), 225–238.

[26] D. Roelants van Baronaigien and F. Ruskey, A Hamilton path in the rotation lattice of binary trees, *Congressus Numerantium* **59** (1987), 313–318.

[27] D. Roelants van Baronaigien and F. Ruskey, Generating t-ary trees in A-order, *Information Processing Letters* **27** (1988), 205–213.

[28] D. Roelants van Baronaigien, A loopless algorithm for generating binary tree sequences, *Information Processing Letters* **39** (1991), 189–194.

[29] D. Roelants van Baronaigien, A loopless Gray-code algorithm for listing k-ary trees, *Journal of Algorithms* **35** (2000), 100–107.

[30] F. Ruskey and T. C. Hu, Generating binary trees lexicographically, *SIAM Journal on Computing* **6** (1977), 745–758.

[31] F. Ruskey, Generating t-ary trees lexicographically, *SIAM Journal on Computing* **7** (1978), 424–439.

[32] F. Ruskey and A. Proskurowski, Generating binary trees by transpositions, *Journal of Algorithms* **11** (1990), 68–84.

[33] C. D. Savage, A survey of combinatorial Gray codes, *SIAM Review* **39** (1997), 605–629.

[34] T. Takaoka, O(1) time algorithms for combinatorial generation by tree traversal, *The Computer Journal* **42** (1999), 400–408.

[35] A. E. Trojanowaki, Ranking and listing algorithms for k-ary trees, *SIAM Journal on Computing* **7** (1978), 492–509.

[36] V. Vajnovszki, On the loopless generation of binary tree sequences, *Information Processing Letters* **68** (1998), 113–117.

[37] V. Vajnovszki, Generating a Gray code for P-sequences, *Journal of Mathematical Modelling and Algorithms* **1** (2002), 31–41.

[38] S. G. Williamson, *Combinatorics for Computer Science*, Computer Science Press, Rockville, MD, 1985.

[39] Ro-Yu Wu, Jou-Ming Chang, Yue-Li Wang, The Gray-code graph of t-ary trees using RD-sequences is Hamiltonian, *Proceedings of the 24rd Workshop on Combinatorial Mathematics and Computation Theory,* 2007, 39–43.

[40] L. Xiang, K. Ushijima, and S. G. Akl, Generating regular k-ary trees efficiently, *The Computer Journal* **43** (2000), 290–300.

[41] L. Xiang, K. Ushijima, and C. Tang, Efficient loopless generation of Gray codes for k-ary trees, *Information Processing Letters* **76** (2000), 169–174.

[42] L. Xiang, K. Ushijima, and C. Tang, On generating k-ary trees in computer representation, *Information Processing Letters* **77** (2001), 231–238.

[43] L. Xiang, K. Ushijima, and Y. Asahiro, Coding k-ary trees for efficient loopless generation in lexicographic order, in: *Proc. of International conference on Information Technology: Coding and Computing* (ITCC'02), IEEE Computer Society Press, Las Vegas, April, 2002, 396–401.

[44] Z. Yongjin and W. Jianfang, Generating k-ary trees in lexicographic order, *Scientia Sinica* **23** (1980), 1219–1225.

[45] S. Zaks, Lexicographic generation of ordered trees, *Theoretical Computer Science* **10** (1980), 63–82.

[46] S. Zaks, Generation and ranking of k-ary trees, *Information Processing Letters* **14** (1982), 44–48.

[47] D. Zerling, Generating binary trees using rotations, *Journal of the ACM* **32** (1985), 694–701.